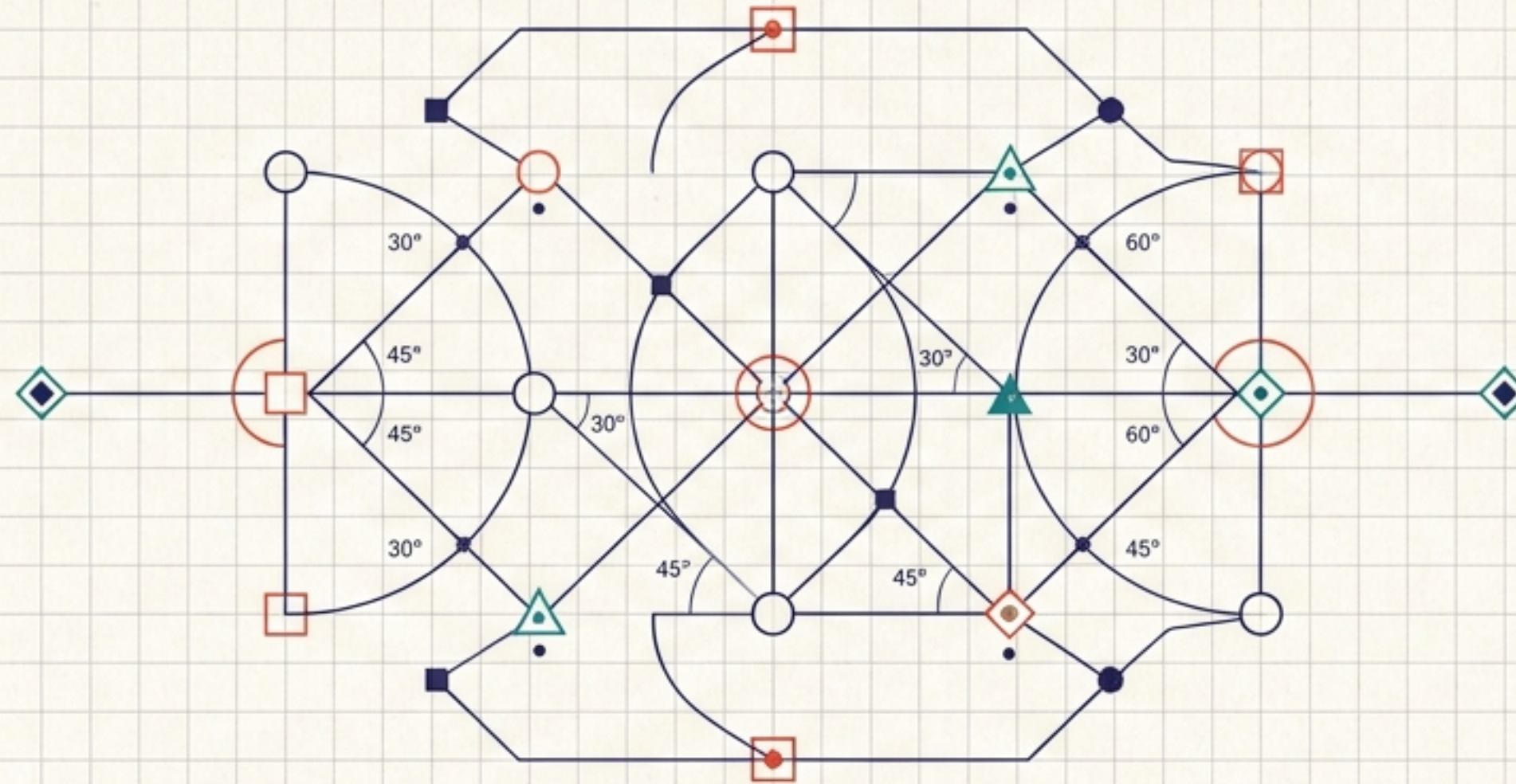
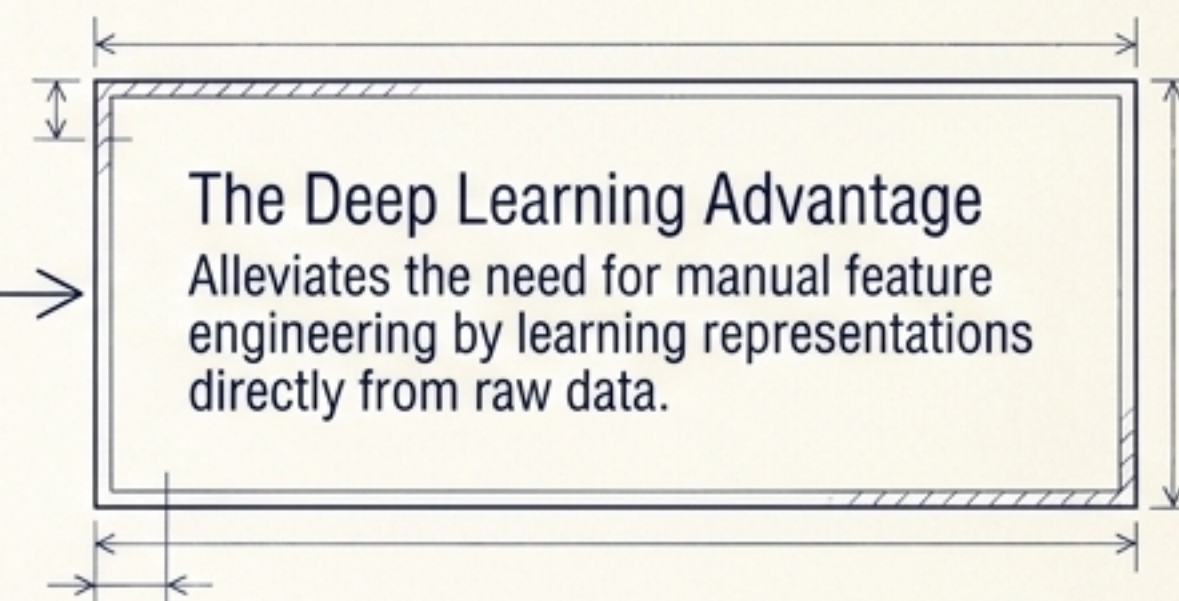
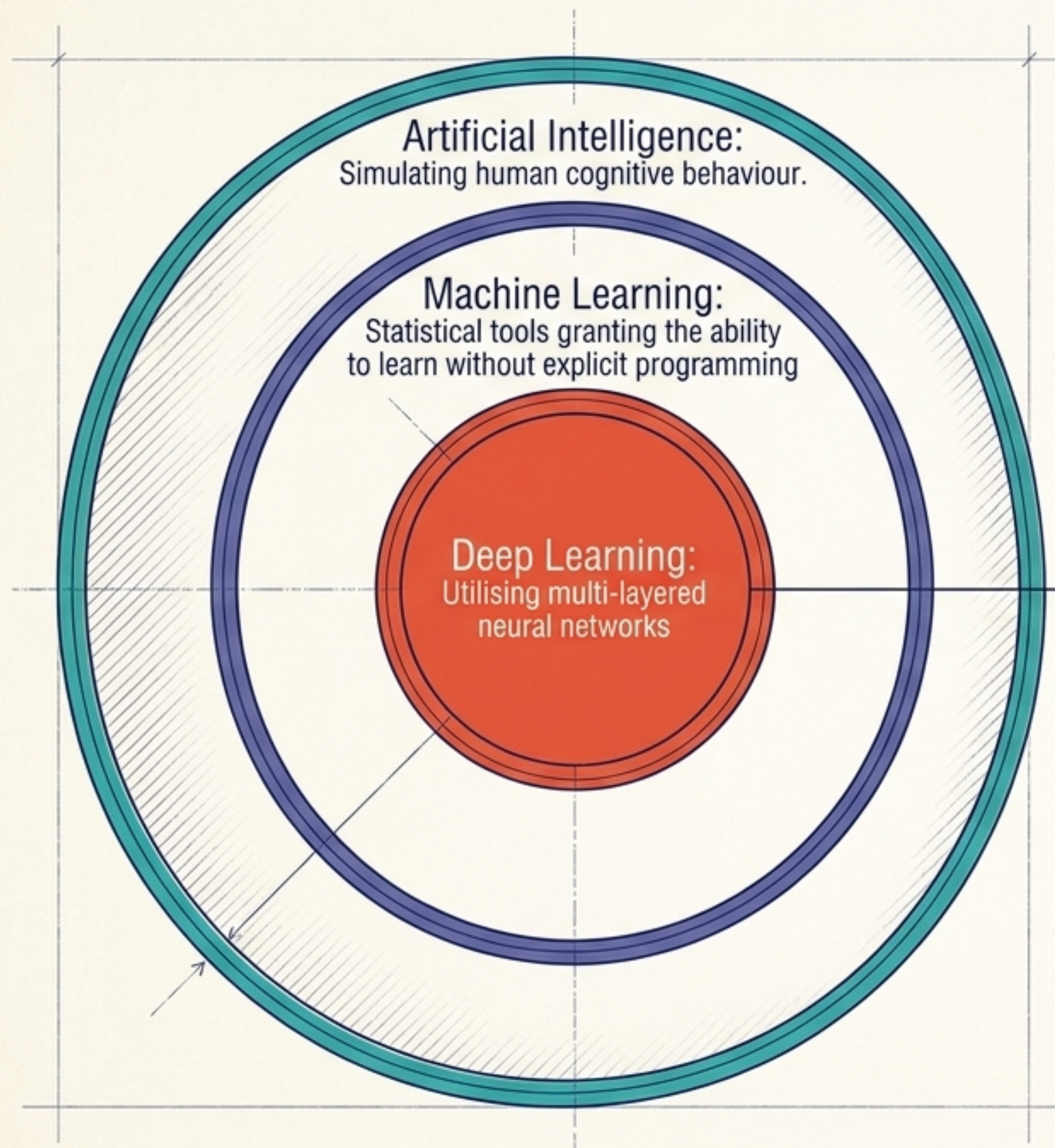




THE NEURAL BLUEPRINT

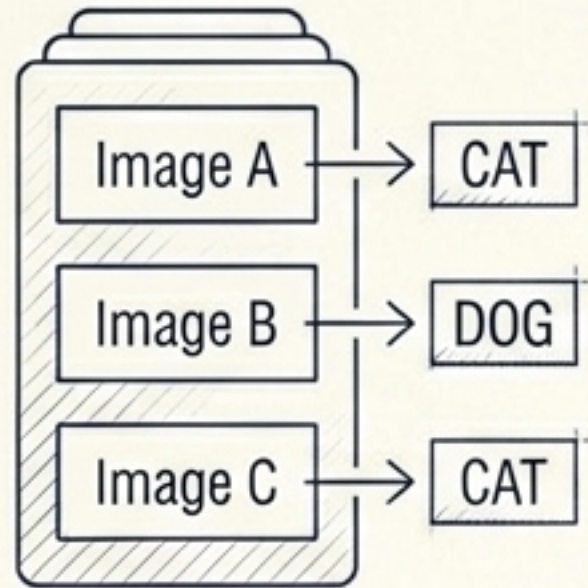
Foundational Mechanics of Machine Learning and Deep Networks

TECHNICAL SYNTHESIS // MACRO TO MICRO



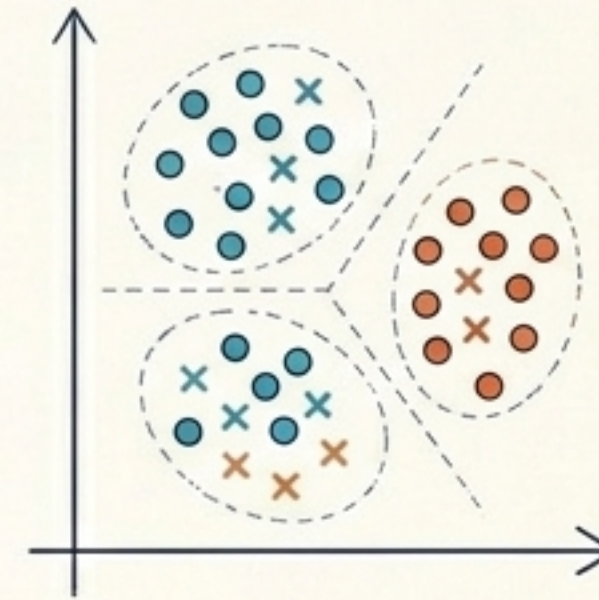
The Two Primary Learning Paradigms

Supervised Learning



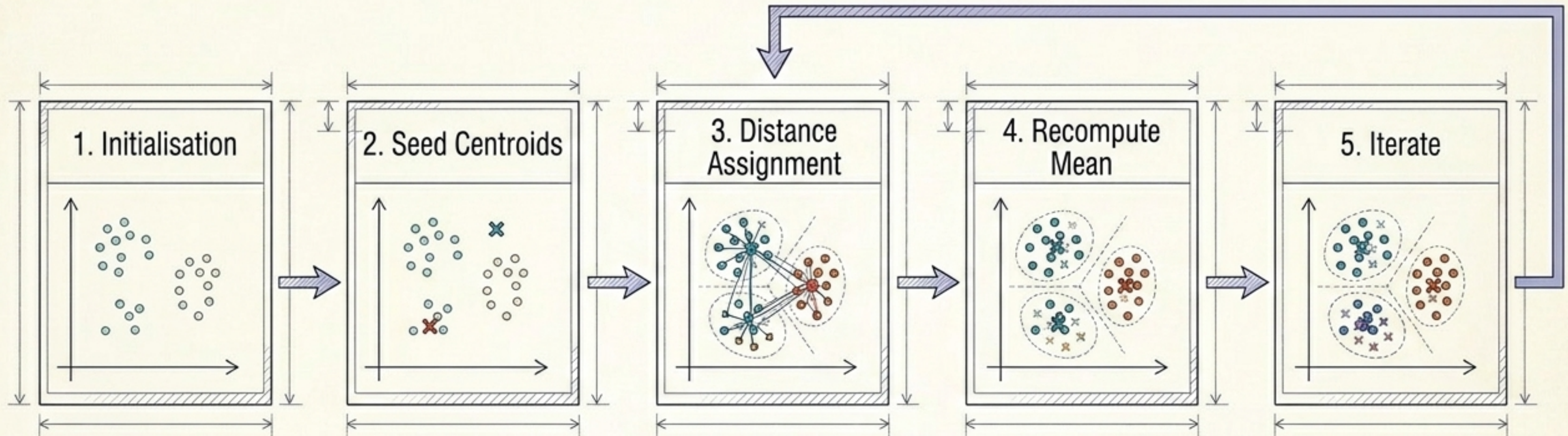
- Data Requirement: Labelled data.
- Goal: Find the rule to predict labels for unseen data.
- Branches:
 - Classification: Predicting a distinct class (e.g., predicting grades based on hours studied).
 - Regression: Predicting a continuous value (e.g., predicting land price based on area).

Unsupervised Learning



- Data Requirement: Unlabelled data.
- Goal: Group data based on features to discover hidden patterns.
- Primary Approach:
 - Clustering: Grouping similar feature vectors (e.g., K-Means algorithm).

The K-Means Clustering Pipeline



1. Initialisation
Choose K (the desired number of clusters).

2. Seed Centroids
Randomly plot K points as initial centroids.
E.g., $C1: (2,2)$ and $C2: (6,8)$.

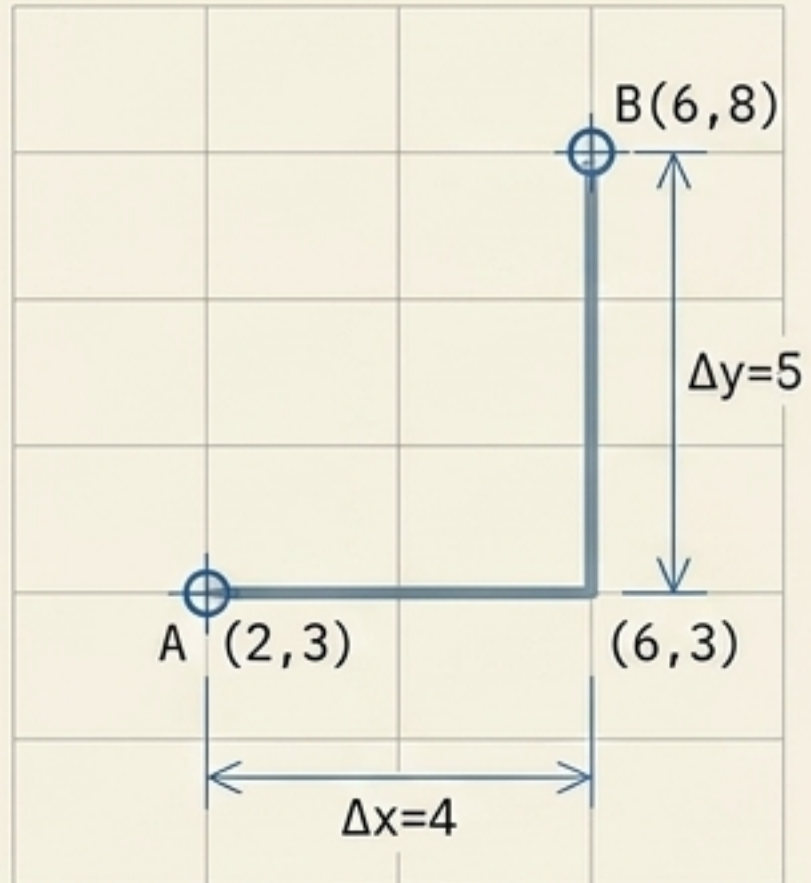
3. Distance Assignment
Compute distance from each data point to the centroids.
Assign points to the nearest centroid.

4. Recompute Mean
Calculate the new centroid by finding the mean coordinates of all points in the newly formed cluster.

5. Iterate
Repeat Steps 3 and 4 until the centroids converge and stop moving.

Measuring Feature Similarity: Two Geometries

Manhattan Distance (L1)



Math:

$$|x_2 - x_1| + |y_2 - y_1|$$

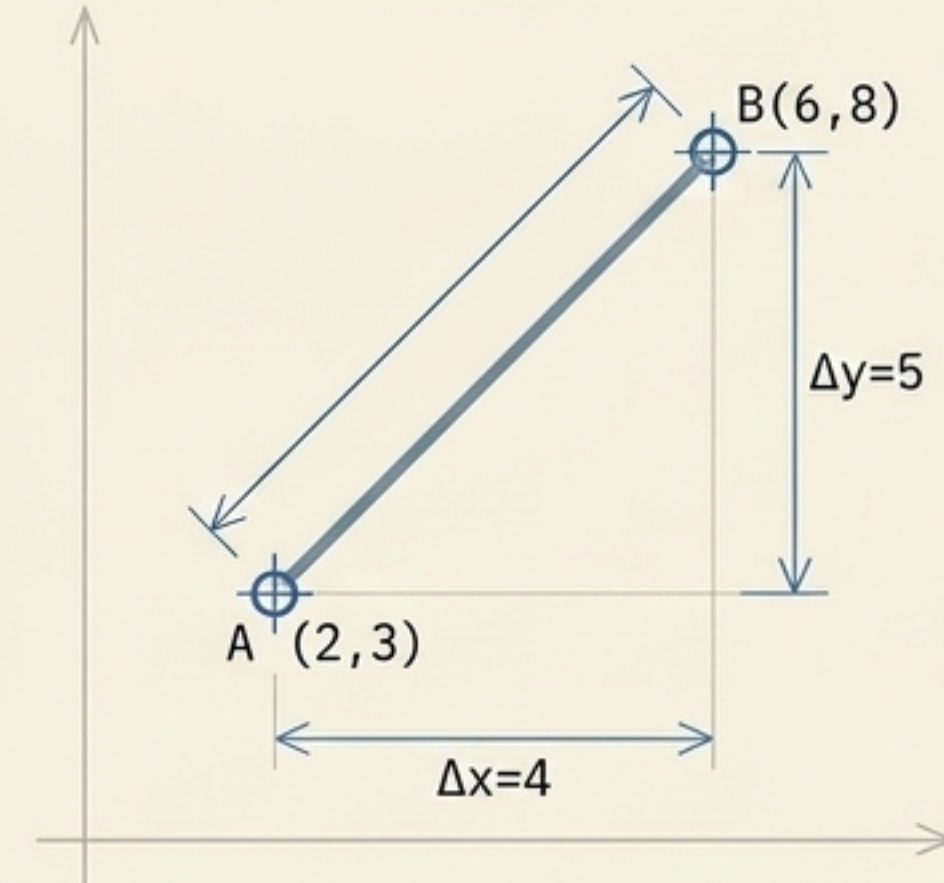
Use Case:

Preferred for high-dimensional feature vectors.

Example Calculation:

$$|6 - 2| + |8 - 3| = 9$$

Euclidean Distance (L2)



Math:

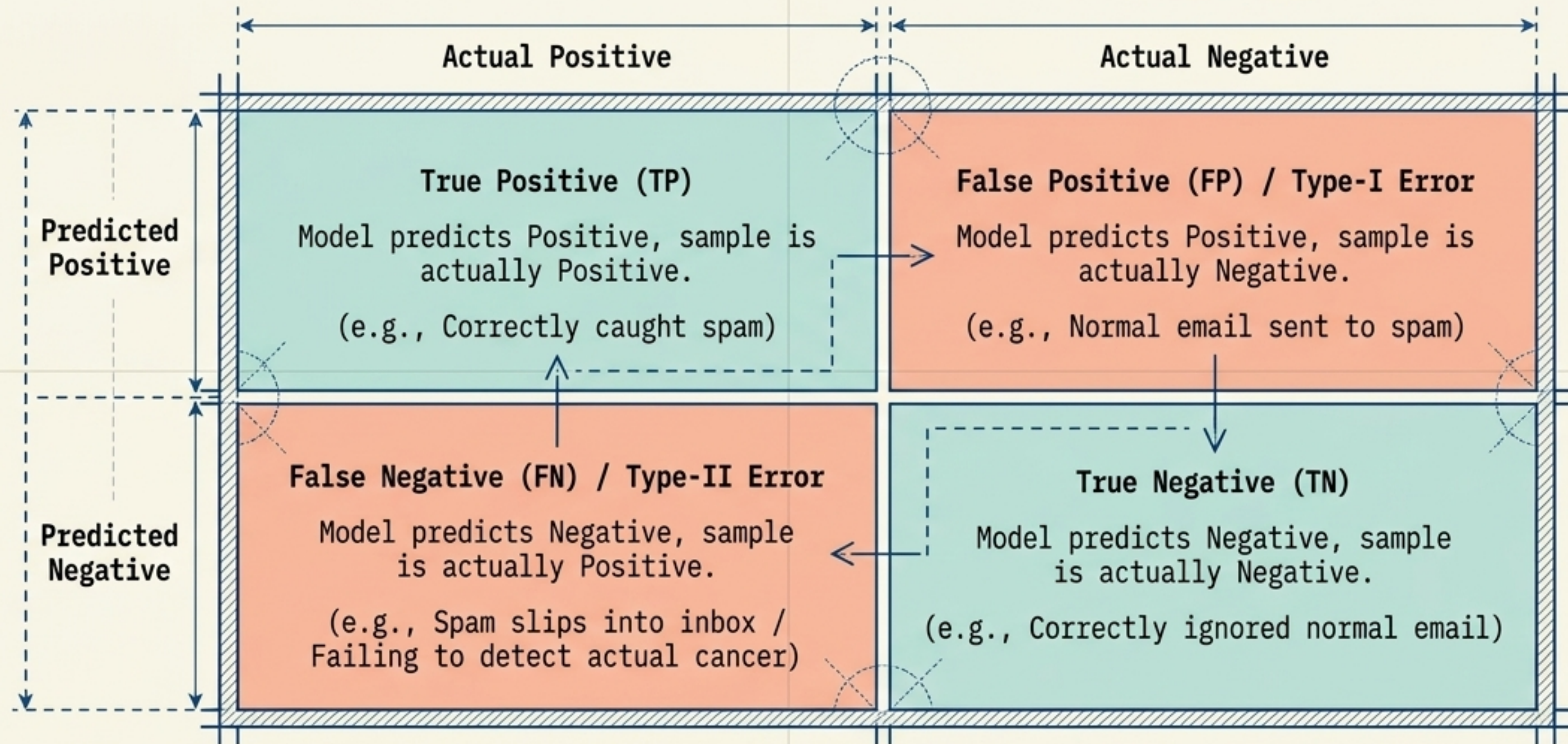
$$\sqrt{((x_2 - x_1)^2 + (y_2 - y_1)^2)}$$

Concept:

The square root of the sum of the squares of coordinate differences. Measures the shortest straight-line path.



Evaluating Predictions: The Confusion Matrix



NOTE: Accuracy ($[TP+TN]/\text{Total}$) can be highly misleading on imbalanced datasets (e.g., achieving 99% accuracy by simply predicting 'no cancer' every time on a skewed dataset).

The Precision vs. Recall Trade-off

FORMULAS & DEFINITIONS

Precision: $TP / (TP + FP)$

Out of all predicted positives, how many were truly positive?

Recall: $TP / (TP + FN)$

Out of all actual positives, how many did we identify?

F1-Score: $(2 \times \text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$

THRESHOLD SLIDER: SPAM FILTER EXAMPLE

Scenario: 100 emails, 20 actual spam

Strict

Relaxed

Strict Threshold ($t=0.9$)

Model predicts only 10 spam.

TP=9, FP=1, FN=11

Precision = 0.90 | Recall = 0.45

Outcome: High precision, but many actual spam emails are missed.

Relaxed Threshold ($t=0.3$)

Model predicts 30 spam.

TP=18, FP=12, FN=2

Precision = 0.60 | Recall = 0.90

Outcome: Catches most spam, but moves many normal emails to spam.

Architecture of a Multi-Layer Perceptron (MLP)

Input Layer (x)

Ingests the raw feature vectors.

x1

x2

x3

h1

h2

h3

h4

h1'

h2'

h3'

h4'

Output Layer (y-hat)

The final prediction or classification.

y-hat1

y-hat2

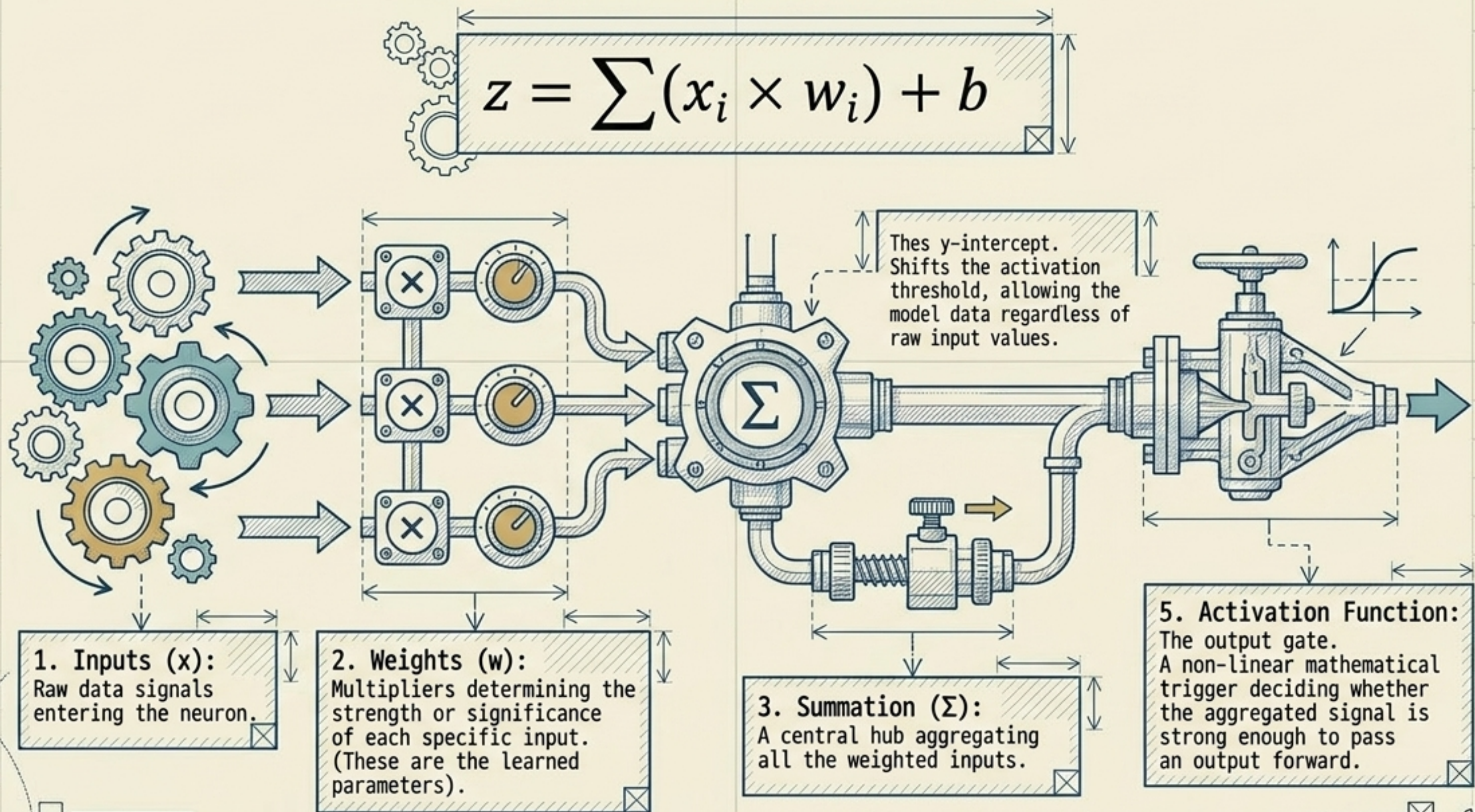
Hidden Layers (h)

The core of deep learning. Multiple neurons learn multiple features in parallel. Earlier layers learn simple patterns; deeper layers combine them into complex, abstract representations.

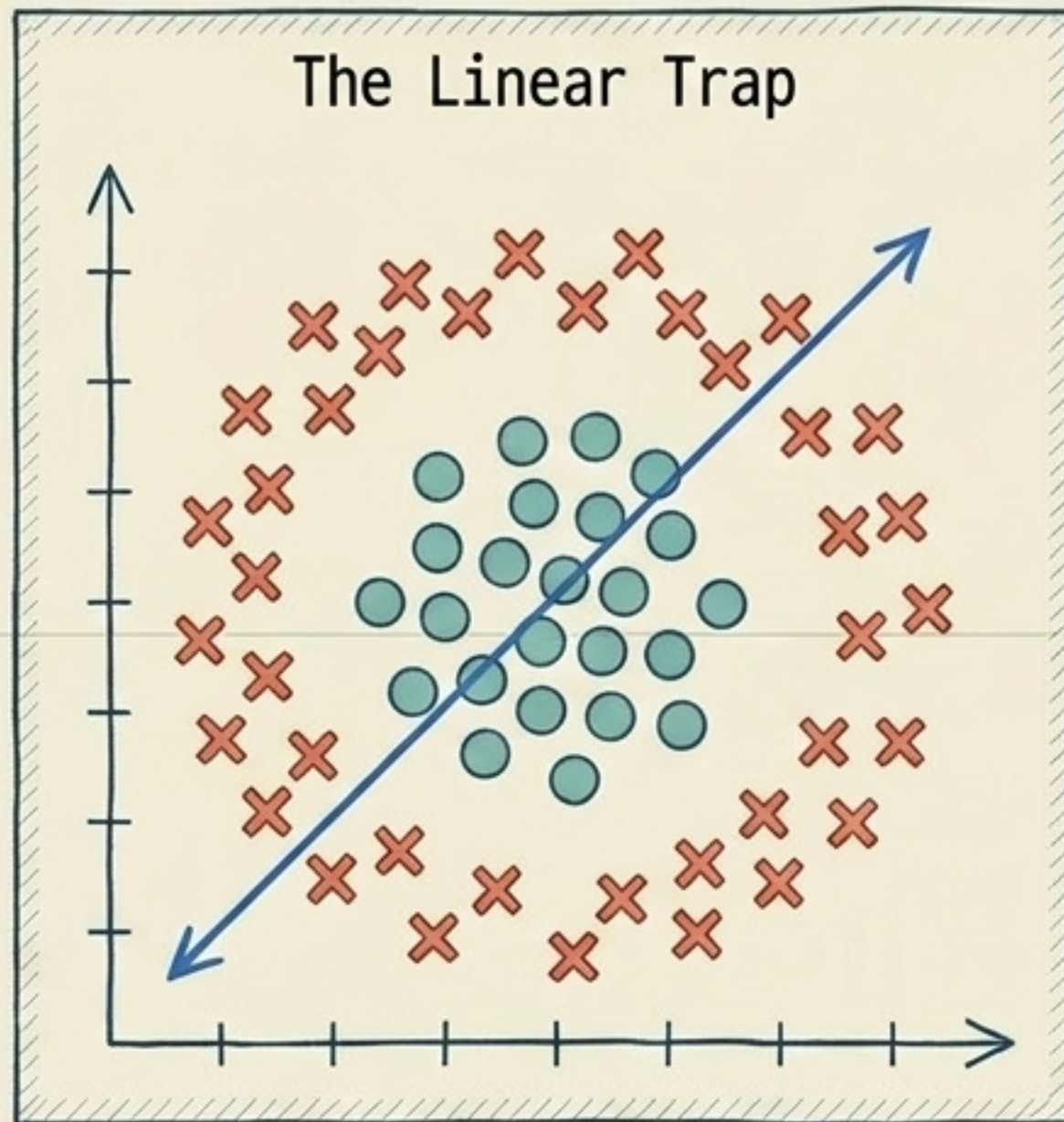
Weight Connections

The learned mathematical significance of specific features.

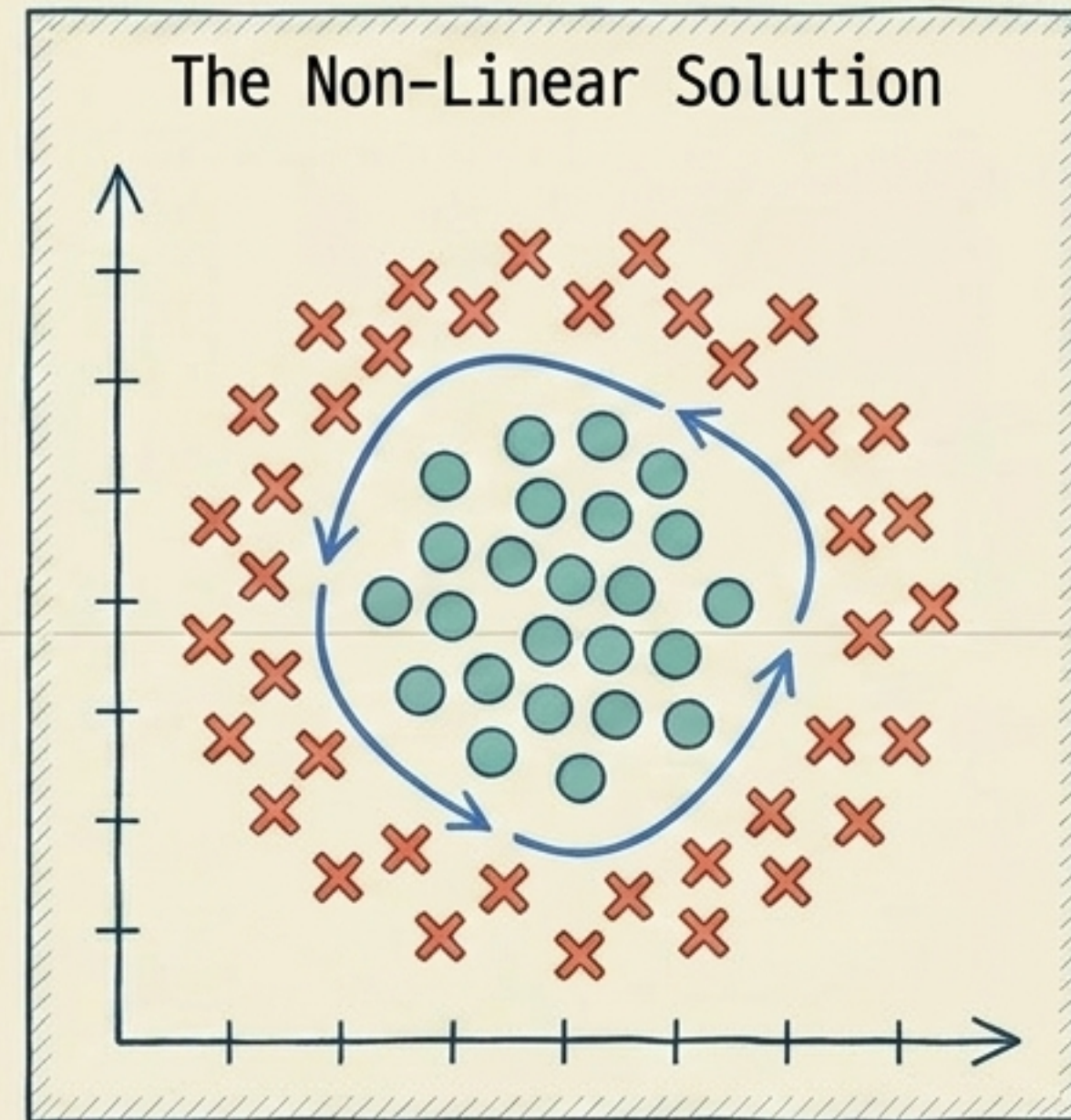
The Atomic Unit: Anatomy of an Artificial Neuron



The Necessity of Non-Linearity



Without an activation function, a neuron outputs a straight line ($y = mx + b$). Stacking multiple linear layers still mathematically collapses into a single linear model. It cannot capture real-world complexity.

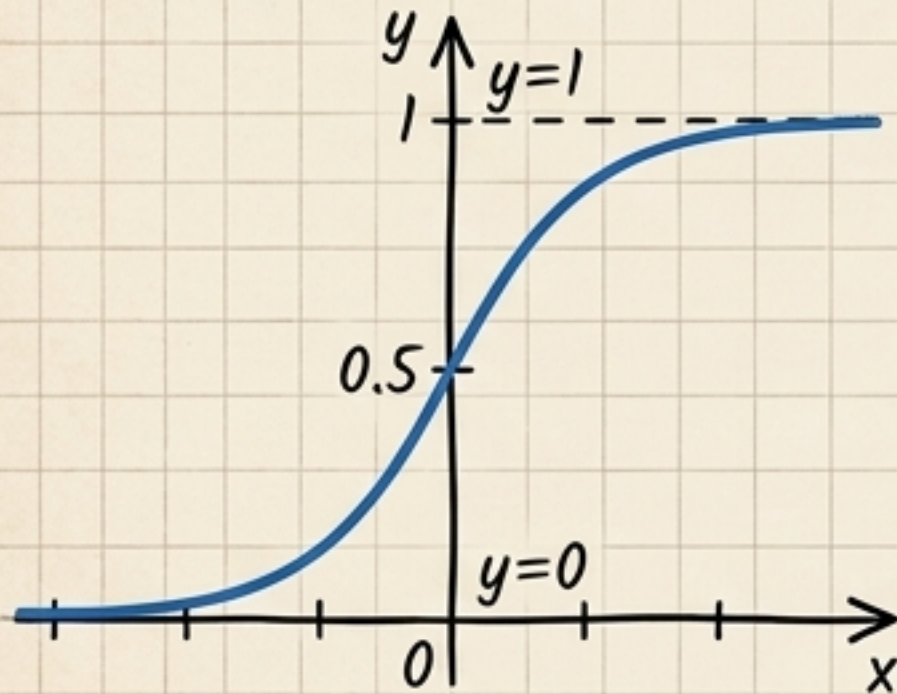


Activation functions introduce non-linear mathematical behaviour, granting neural networks the geometric flexibility to bend, curve, and mould around highly complex data distributions.

The Activation Arsenal

Sigmoid

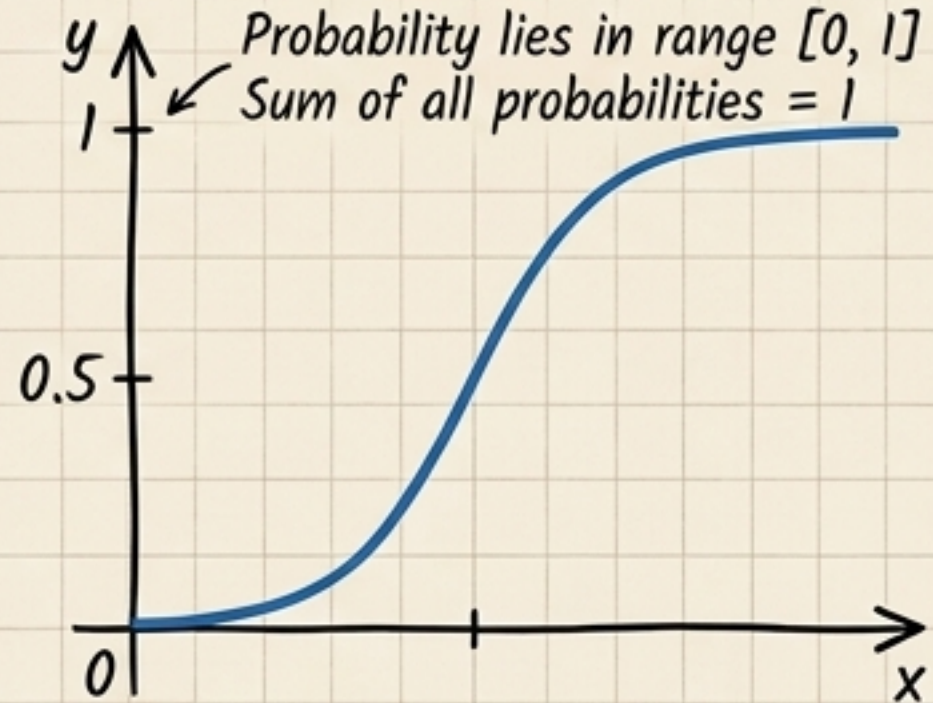
$$\sigma(z) = \frac{1}{1 + e^{-z}}$$



Use Case: Binary Classification.
Outputs a probability.
If < 0.5 , predict Class 0.
If ≥ 0.5 , predict Class 1.

Softmax

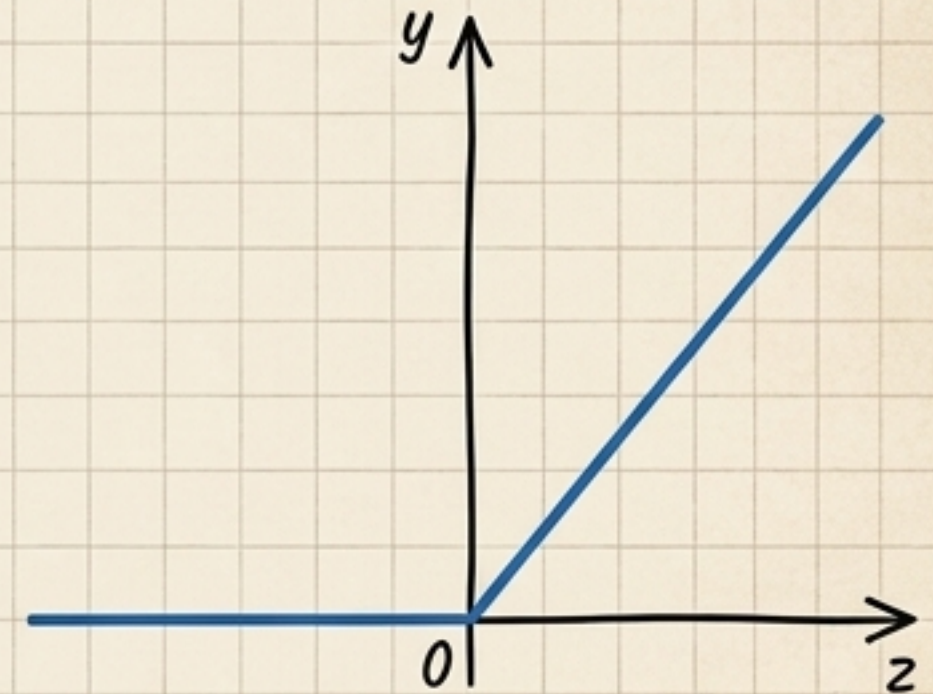
$$\frac{e^{z_i}}{\sum e^{z_j}}$$



Use Case: Multi-class classification.
Converts raw, unbounded logits into a
standardised probability distribution
where all outputs sum perfectly to 1.

ReLU (Rectified Linear Unit)

$$\max(0, z)$$



Use Case: Hidden layers.
Highly computationally efficient.
Addresses the vanishing gradient
problem and introduces deliberate
sparsity.

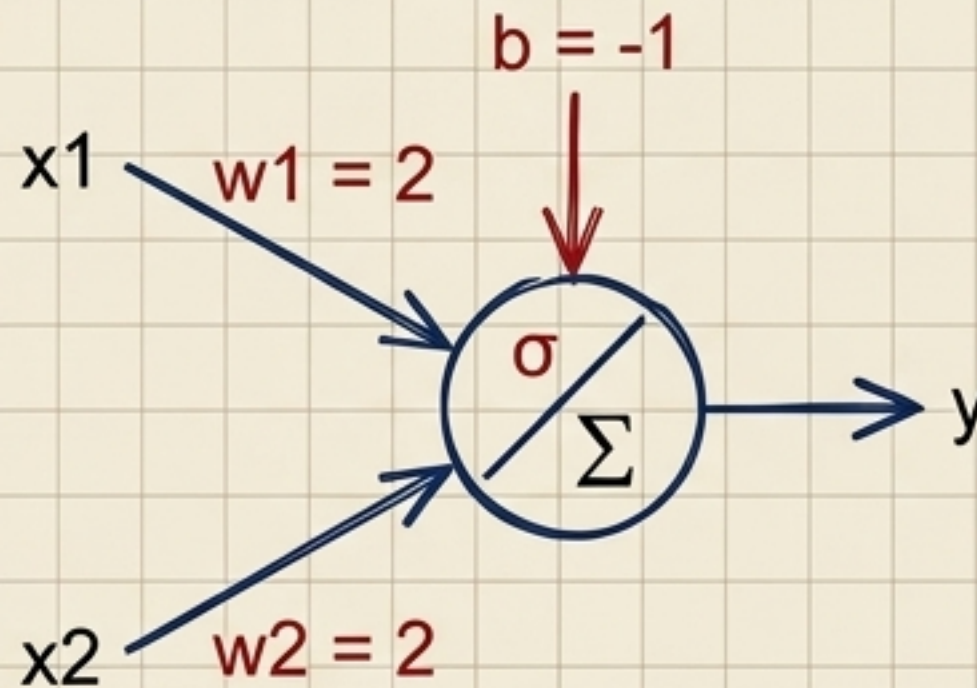
Sigmoid in Action: Modelling an OR Gate

The Goal

x1	x2	y
0	0	0
1	0	1
0	1	1
1	1	1

Combine inputs so the neuron fires (outputs > 0.5) if either input is 1.

The Architecture



$$y = \sigma(x_1w_1 + x_2w_2 + b)$$

The Mathematical Proof

If $x_1=0$, $x_2=0$:

$$\begin{aligned}\text{Output} &= \sigma(0*2 + 0*2 - 1) \\ &= \sigma(-1) < 0.5 \rightarrow 0\end{aligned}$$

If $x_1=1$, $x_2=0$:

$$\begin{aligned}\text{Output} &= \sigma(1*2 + 0*2 - 1) \\ &= \sigma(2 - 1) = \sigma(1) > 0.5 \rightarrow 1\end{aligned}$$

Takeaway: By simply adjusting weights and bias, a single non-linear neuron easily learns specific logical behaviours.

Softmax in Action: Predicting Outcomes

Scenario: Predicting a student's grade (A, B, or C) based on hours studied.

Step 1: Raw Logits (z)

The network outputs arbitrary, unbounded values for each class.

Grade A: 27.0

Grade B: 34.1

Grade C: 16.7

Step 2: Exponentiate

Raise "e" to the power of each logit. This eliminates negative numbers and exponentially scales the values.

$$e^{27.0} \mid e^{34.1} \mid e^{16.7}$$

Step 3: Normalise

Divide each exponentiated value by the total sum of all values.

Result: A probability distribution. Grade B yields $\approx 99.98\%$.
The model confidently predicts the class with the highest probability.

The Learning Engine: Backpropagation

1. The Forward Pass (Prediction)

Raw data flows through the network layers. Neurons calculate weighted sums, apply activation functions, and generate an estimated output prediction (y_e).

2. Error Calculation

The system measures the mathematical gap between the prediction and reality.

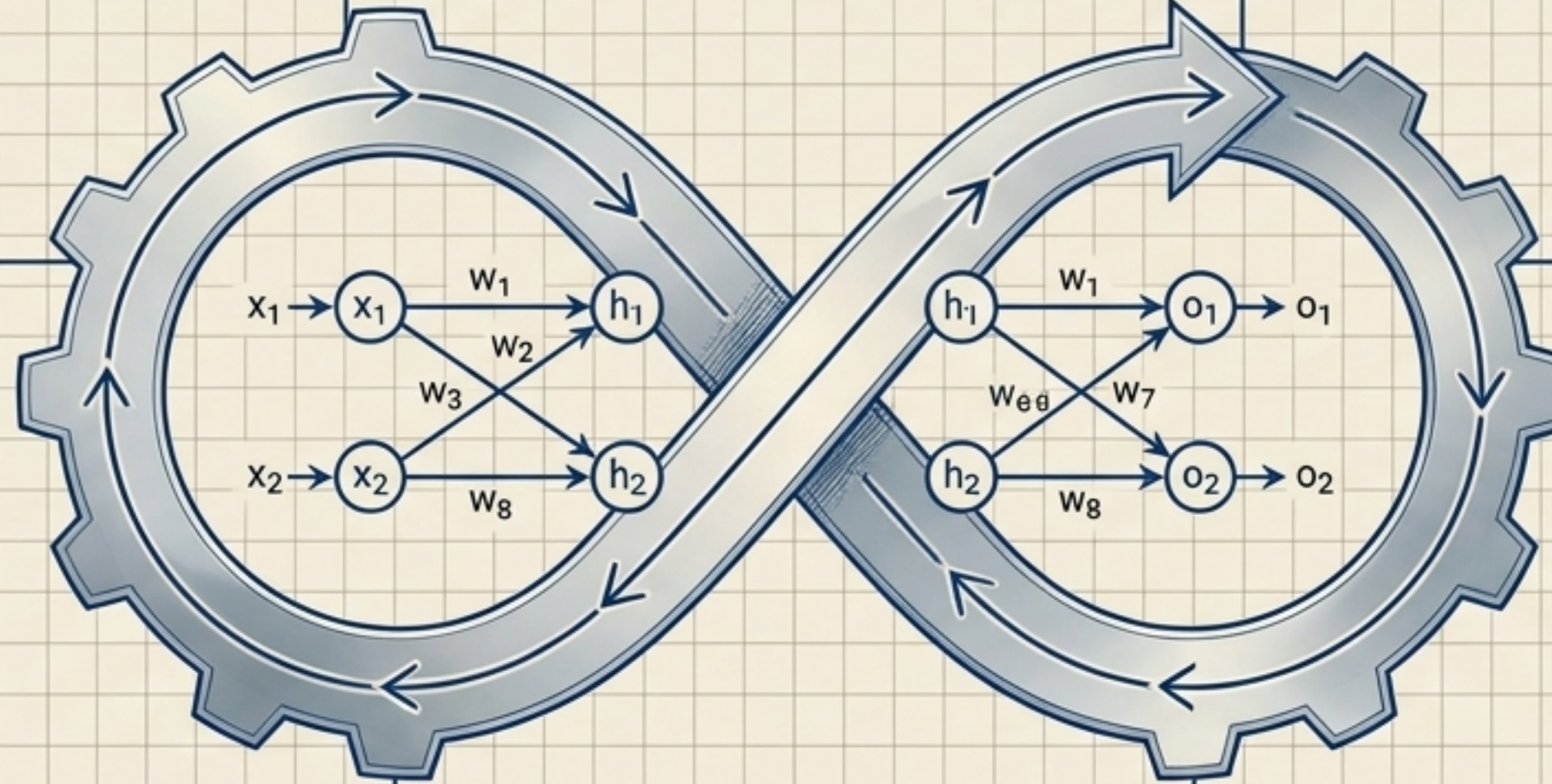
$$\text{Error} = \text{Actual } (y_{pp}) - \text{Estimated } (y_{ee})$$

3. The Backward Pass (Correction)

Using chain-rule derivatives, the network calculates the gradient—exactly how much each individual weight contributed to the overall error.

4. Weight Update

Parameters are updated. If the prediction was too high, contributing weights are decreased. The cycle repeats continuously until the error is minimised.



Gradient Descent: Navigating the Error Landscape

Batch Gradient Descent

Mechanics: Computes the gradient using the ENTIRE training dataset before updating weights once.

Pros/Cons: Highly stable descent, but extremely slow for large datasets and prone to getting stuck in local minima.

Stochastic Gradient Descent (SGD)

Mechanics: Selects a SINGLE random data point to compute the gradient and update weights.

Pros/Cons: Very fast and escapes local minima easily, but causes high oscillation and erratic instability.

Mini-Batch Gradient Descent

Mechanics: Divides the dataset into smaller subsets. Updates happen after each batch.

The Goldilocks Solution: Balances the computational efficiency of SGD with the convergence stability of Batch descent.

